

# REST 練習 3：JSON 序列化與集合查詢

## Lecture

REST API 的 Response Body 需要一個雙方都懂的資料格式，這就是 **Content-Type** header 的作用：告訴呼叫端「這個 Body 要用什麼方式解讀」。現代 Web API（包含這門課）幾乎都選 JSON（**application/json**）當作預設格式——比 XML 輕量、幾乎所有程式語言都有現成的 JSON 解析函式庫、瀏覽器端的 JavaScript 原生就懂 JSON。

ABAP 不是天生就懂 JSON，資料在 ABAP 內部一直是 Structure/Internal Table，要轉成 JSON 字串才能放進 Response Body——**/UI2/CL\_JSON** 就是 SAP 內建的 ABAP ↔ JSON 轉換工具類別（注意它不是 Classic REST 框架的一部分，是一個獨立的、任何 ABAP 程式都能用的通用工具）：

- **SERIALIZE**：ABAP 資料 → JSON 字串（這題會用到）
- **DESERIALIZE**：JSON 字串 → ABAP 資料（rs06 會用到）
- **PRETTY\_NAME** 參數控制欄位名稱的大小寫轉換：ABAP 的欄位名稱習慣全大寫底線分隔（**CARRID**），但 JSON / JavaScript 世界習慣 camelCase（**carrId**）；這門課統一用 **PRETTY\_MODE-CAMEL\_CASE**，之後每一題的 JSON 輸出/輸入都遵循這個慣例

這題也是本課程第一次示範「集合查詢」：把一個 Internal Table（多筆 **SFLIGHT**）整個丟給 **SERIALIZE**，直接得到一個 JSON 陣列——不用自己寫迴圈組字串。

## 學習目標

- 理解為什麼純文字（\t 分欄）不是給前端消費的好格式，JSON 才是 REST API 事實上的標準交換格式
- 會用 **/UI2/CL\_JSON=>SERIALIZE()** 把內部表格直接轉成 JSON 字串，不用手工拼字串
- 理解 **PRETTY\_NAME** 參數的作用：ABAP 欄位名稱是全大寫（**CARRID**），但 JSON 慣例是 camelCase（**carrid** → **carrId**，多字欄位如 **CONNID** 這類單字沒差，但實務上遇到 **MATNR** / 複合欄位時 camelCase 轉換規則要知道去哪查）
- 會設定 **Content-Type: application/json**，讓呼叫端（瀏覽器、前端 fetch）正確解讀回應內容

## 為什麼要換成 JSON

rs02 的 **ZCL\_RS02\_CARRIERS** 回傳 \t 分欄的純文字，呼叫端（例如一支前端 JS）要自己 **split('\t')**、**split('\n')** 才能解析，欄位一多、字串裡剛好出現 tab 或換行字元就會壞掉。JSON 是有結構、有型別、瀏覽器與所有主流語言都有現成 parser 的格式，這就是為什麼幾乎所有 REST API 預設都用 JSON 當交換格式。

SAP 標準已經有 **/UI2/CL\_JSON** 這個類別做序列化 / 反序列化，不需要自己拼字串、也不需要額外裝第三方套件。

## 事前準備

ADT 端物件已由課程準備好（\$TMP）：

- **ZCL\_RS03\_APP**——Application Class，router 掛一個 **/flights** 資源
- **ZCL\_RS03\_FLIGHTS**——查 **SFLIGHT** 前 20 筆，序列化成 JSON 陣列回傳

## 題目需求 ( 對照已建好的答案物件 )

1. **ZCL\_RS03\_APP** 的 **GET\_ROOT\_HANDLER** : 沿用 **rs02** 的 **router** 寫法 , 只是這次只掛一個資源 :

```
``abap DATA(lo_router) = NEW cl_rest_router( ). lo_router->attach( iv_template =  
'/flights' iv_handler_class = 'ZCL_RS03_FLIGHTS' ). ro_root_handler = lo_router. ``
```

1. **ZCL\_RS03\_FLIGHTS** 的 **GET** :

```
``abap SELECT carrid, connid, fldate, price, currency FROM sflight ORDER BY carrid, connid, fldate INTO TABLE  
@DATA(lt_flight) UP TO 20 ROWS.
```

```
DATA(lv_json) = /ui2/cl_json=>serialize( data = lt_flight pretty_name = /ui2/cl_json=>pretty_mode-camel_case  
).
```

```
DATA(lo_entity) = mo_response->create_entity( ). lo_entity->set_string_data( lv_json ). lo_entity-  
>set_content_type( if_rest_media_type=>gc_appl_json ). `` 注意 UP TO 20 ROWS 要接在 INTO TABLE  
@DATA(...) 之後 , 不能直接接在 ORDER BY 後面 ( 語法錯誤 "UP" is not allowed here. , 這是 Open SQL 子  
句順序的常見坑 , ORDER BY 一定要寫在 INTO 之前 , UP TO n ROWS 一定要寫在 INTO 之後 ) 。
```

## SICF 掛載步驟 ( 比照 rs01/rs02 , 只是換掉數值 )

沿用既有的 **/sap/bc/zrest\_training** 分類節點 , 這題只要在其底下再加一個子節點 :

1. SICF 交易碼 , 在 **zrest\_training** node 上右鍵 → **New Sub-Element** , Service Name 填 **rs03**
2. Handler List 掛 **ZCL\_RS03\_APP**
3. Activate Service
4. 測試路徑 :

- **https://erpdemo01.itts.com.tw:44300/sap/bc/zrest\_training/rs03/flights?sap-client=130**

⚠ **2026-08-02 更正** : 原本以為連不到 SAP Host 內網時才需要改用 **SPROX\_HTTP\_REQUEST** ——已實測推翻 , 只要用系統對外的正確主機名稱 + Port ( 不是內網固定 IP ) , 瀏覽器/Postman 在外網一樣直接連得到 , 不需要等內網 / VPN 。 **SPROX\_HTTP\_REQUEST** 依然是可用的替代測試方式 ( 例如懶得開瀏覽器、或想在 ABAP 端直接看 Response ) , 但不是「外網限定才需要」的 workaround 。 注意 **URL** 一樣要填完整網址 ( 含 **http://** 或 **https://** 與主機:port ) , 不能只填 **/sap/bc/...** 相對路徑——**SPROX\_HTTP\_REQUEST** 雖然是在 Application Server 上跑的 ABAP 程式 , 但它執行的是一支真正的 HTTP request ( 呼叫 ICM 的 HTTP Port ) , 不是程式內部呼叫 , 沒給 host:port 會找不到這個 API endpoint ( 跟瀏覽器測試用的是同一組完整 URL ) 。

## 預期輸出 ( 範例 )

```
[  
  {"carrid":"AA","connid":17,"fldate":"2018-10-29","price":422.94,"currency":"USD"},  
  {"carrid":"AA","connid":17,"fldate":"2018-10-30","price":422.94,"currency":"USD"},  
  ...  
]
```

注意兩個容易誤判的地方 ( 2026-07-12 實際打過才確認 , 不是憑空猜的 ) :

- `connid` 是 `NUMC` 型別，`/UI2/CL_JSON` 預設會轉成不帶引號的數字、去掉前導 0 ( `17`，不是 `"0017"` )
- `fldate` 是 `DATS` 型別，`/UI2/CL_JSON` 固定序列化為 `YYYY-MM-DD` ( `2018-10-29` )，不是原始的 8 碼數字字串 `20181029`，這點不受 `PRETTY_NAME` 參數影響

這個「輸出格式」跟 `rs04` 單筆查詢「輸入格式」( URL 路徑要用 8 碼數字 `20181029`，不能帶短橫線 ) 不對稱，是下一題會踩到的坑，先在這裡點出來。

## 團隊實務備註

- `/UI2/CL_JSON=>SERIALIZE` 是唯讀轉換，不會動到原本的 `lt_flight`；對稱的 `/UI2/CL_JSON=>DESERIALIZE` 留到 `rs06` ( `POST` 建立資源 ) 才會用到
- `PRETTY_NAME` 除了 `CAMEL_CASE` 還有 `LOW_CASE` ( 全小寫，不做駝峰轉換 )、`EXTENDED` ( 更聰明的縮寫辨識，如 `ID` 不會被拆開 )——實務上通常固定用一種，跟前端團隊講好就好，不要每個 API 各用各的
- `SET_CONTENT_TYPE` 沒設的話，`CL_REST_RESOURCE` 預設 `Content-Type` 不一定是 `JSON`，瀏覽器/Postman 可能會把回應當純文字顯示、或前端 `fetch().json()` 解析失敗，這是最容易漏掉的一步

## 思考題

1. `SFLIGHT-FLDATE` 是 `DATS` 型別，`/UI2/CL_JSON` 序列化後會變成什麼樣的字串？如果前端要用 JavaScript `Date` 物件解析，這個格式夠用嗎？( 提示：`SERIALIZE` 有 `TS_AS_ISO8601` 參數，但那是給 `Timestamp` 用的，`DATS` 本身要另外處理 )
2. 承 `rs02` 的思考題 2：現在 `/flights` 回傳的是 `JSON array`，如果呼叫端想知道「總共查到幾筆」而不是自己數陣列長度，這個設計要怎麼調整？( 提示：外層再包一層 `{ "count": ..., "data": [...] }` 的物件，而不是直接回傳陣列——這也是 `rs09` 期末整合會遇到的設計選擇 )
3. `UP TO 20 ROWS` 是寫死在程式碼裡的，如果之後想讓呼叫端自己決定要幾筆 ( 例如 `?limit=50` )，要透過什麼機制取得 URL 上的參數？( 這正是 `rs05 Query Parameter` 篩選要教的 )

## 答案

見 `zcl_rs03_app.clas.abap`、`zcl_rs03_flights.clas.abap` ( `SAP` 端物件 `ZCL_RS03_APP / ZCL_RS03_FLIGHTS` )。SICF Service 路徑 `/sap/bc/zrest_training/rs03`。